

[Name der Hochschule]

[Art der Arbeit]

[Titel der Fallstudie zum Technologie-Template]

[Untertitel]

[Aufgabenstellung]

Kurs: [Kurs / Modul]
Studiengang: [Studiengang]
Autor: [Vorname Nachname]
Matrikelnummer: [Matrikelnummer]
Tutor: [Tutor / Betreuer]
Abgabedatum: [Monat Jahr]

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	1
1.4 Fragestellungen	2
1.5 Methodisches Vorgehen	2
1.6 Aufbau der Fallstudie	3
2 Anforderungen an das Technologie-Template	4
2.1 Zielbild des Technologie-Templates	4
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	4
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	5
2.4 Qualitäts- und Wartbarkeitsanforderungen	5
2.5 Abgrenzung	5
3 Architektur des Technologie-Templates	6
3.1 Gesamtarchitektur	6
3.2 Frontend mit Vite und TypeScript	7
3.3 Backend mit FastAPI	8
3.4 Desktop-Integration mit Tauri und Rust	8
3.5 Shared Contracts und Schnittstellen	8
3.6 Konfigurationsmodell	8
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	9
4 Projektprofile und Feature-Modell	9
4.1 Grundprinzip des Profilmodells	9
4.2 Profil Web-only	11
4.3 Profil Web-cloud	11

4.4	Profil Desktop-local	11
4.5	Profil Desktop-cloud	11
4.6	Profil Full-platform	11
4.7	Optionale Capabilities	11
4.8	Single-Repository-Strategie	11
5	Tooling und Entwicklungsworkflow	11
5.1	Rolle von control.py	11
5.2	Projektinitialisierung und Generierung	12
5.3	Systemprüfung und Installation	12
5.4	Entwicklungsbetrieb	12
5.5	Tests und Builds	13
5.6	Release- und Packaging-Workflow	13
5.7	Entwickler-Onboarding	13
6	Qualitätssicherung und Verifikation	13
6.1	Teststrategie	13
6.2	Continuous Integration	14
6.3	Abnahme und technische Verifikation	14
6.4	Reproduzierbarkeit	14
6.5	Security und Konfigurationsgrenzen	14
6.6	Extern verbleibende Prüfungen	14
7	Bewertung des Technologie-Templates	14
7.1	Produktivitätswirkung	14
7.2	Stärken	14
7.3	Schwächen und Grenzen	15
7.4	Wartungsaufwand	15
7.5	Vergleich mit alternativen Template-Strategien	15
7.6	Gesamtbewertung	15
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	15
8.1	Geeignete Projekttypen	15
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	15
8.3	Analyse von Kundenanforderungen	15

8.4	Auswahl des Projektprofils	15
8.5	Generierung eines Kundenprojekts	15
8.6	Überführung in die Produktentwicklung	16
9	Schlussbetrachtung	16
9.1	Zusammenfassung der Ergebnisse	16
9.2	Beantwortung der Fragestellungen	16
9.3	Kritische Würdigung	16
9.4	Ausblick	16
9.5	Fazit	16
	Literaturverzeichnis	17

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	3
2	Gesamtarchitektur des Technologie-Templates	6
3	Komponenten und Verantwortungsgrenzen im Master Repository	6
4	Profilabhängige Laufzeitarchitektur	7
5	Konfigurationspriorität und Sicherheitsgrenzen	8
6	Profilmodell auf Basis eines gemeinsamen Master-Templates	9
7	Strukturelle Feature-Zuordnung der Projektprofile	9
8	Entscheidungsstruktur für die Auswahl eines Projektprofils	10
9	Befehlsgruppen des zentralen Projekt-Toolings	11
10	Workflow der profilgesteuerten Projektgenerierung	12
11	Grundstruktur des Entwicklungsworkflows	12
12	Providerneutrale Deployment-Architektur	13
13	Plattformübergreifendes Modell für Desktop-Releases	13
14	Nachweiskette der technischen Verifikation	13
15	Struktur der profilbezogenen Verifikationsmatrix	14
16	Offene Bewertungsstruktur für die Projekteignung	15
17	Prozess vom Kundenbedarf bis zur Auslieferung	15

Tabellenverzeichnis

1	Zentrale Anforderungen an das Technologie-Template	4
2	Anforderungsmatrix der unterstützten Projektarten	5
3	Struktur zur Dokumentation des Technologie-Stacks	7
4	Struktur zur Dokumentation der Projektprofile	10
5	Struktur zur Dokumentation der technischen Verifikation	14
6	Struktur zur Gegenüberstellung von Stärken und Grenzen	14
7	Struktur zur Bewertung der Projekteignung	15

Abkürzungsverzeichnis

Abk. Bedeutung

1 Einleitung

1.1 Ausgangslage

Die Einrichtung neuer Softwareprojekte erfordert wiederkehrende Entscheidungen zu Architektur, Technologien, Schnittstellen, Konfiguration sowie Test-, Build- und Deployment-Prozessen. Werden diese Grundlagen für jedes Vorhaben unabhängig erstellt, entstehen unterschiedliche technische Ausgangslagen. Web-, Cloud- und Desktop-Anwendungen stellen zudem verschiedene Anforderungen an Komponenten und Bereitstellung. Bei plattformübergreifenden Projekten müssen gemeinsame Bestandteile deshalb klar von spezifischen Erweiterungen getrennt werden.

Eine standardisierte technische Basis kann wiederkehrende Entscheidungen bündeln und ein konsistentes Entwicklungsmodell bereitstellen. Gegenstand der Fallstudie ist das Projekt „Template-Projekte“, ein wiederverwendbares Full-Stack-Technologie-Template für Web-, Cloud- und Desktop-Projekte. Seine Ausgestaltung und Eignung werden im weiteren Verlauf systematisch untersucht (kleiveist, 2026d).

1.2 Problemstellung

Die wiederholte Initialisierung technisch ähnlicher Projekte erzeugt Aufwand außerhalb der produktspezifischen Entwicklung. Ohne gemeinsame Baseline können sich Projektstruktur, Werkzeugauswahl, Konfiguration und Build-Prozesse trotz vergleichbarer Anforderungen unterscheiden. Getrennt gepflegte Ausgangsstände für Web-, Cloud- und Desktop-Anwendungen erhöhen zudem den Wartungsaufwand und begünstigen auseinanderlaufende Technologie- und Dokumentationsstände.

Die zentrale Herausforderung besteht darin, Einheitlichkeit und Flexibilität zu verbinden. Das Template muss genügend gemeinsame Struktur für Standardisierung und Wiederverwendung vorgeben, darf ein Projekt jedoch nicht mit unnötigen Komponenten belasten. Untersucht wird daher, wie eine gemeinsame Basis, Projektprofile und optionale Erweiterungen diesen Zielkonflikt adressieren und wo Anpassungsbedarf verbleibt (kleiveist, 2026a, 2026b).

1.3 Zielsetzung

Ziel der Fallstudie ist die strukturierte Untersuchung des Projekts „Template-Projekte“. Analysiert werden die Architektur, die Aufteilung in gemeinsame und profilspezifische Bestandteile sowie die unterstützenden Entwicklungs- und Qualitätssicherungsprozesse (kleiveist, 2026a, 2026e).

Auf dieser Grundlage werden Wiederverwendbarkeit, Standardisierung und das Produktivitätspotenzial des Templates sowie seine Eignung für unterschiedliche Projektarten bewertet. Die Untersuchung stützt sich auf vorhandene Projektartefakte und Verifikationsnachweise. Daraus sollen Einsatzfelder und Auswahlkriterien für zukünftige Kundenprojekte sowie technische Grenzen und externe Voraussetzungen abgeleitet werden (kleiveist, 2026c). Eine universelle Eignung für sämtliche Softwareklassen oder eine produktspezifische Facharchitektur wird nicht beansprucht.

1.4 Fragestellungen

Aus der beschriebenen Problemstellung und Zielsetzung ergeben sich sechs zentrale Untersuchungsfragen. Sie strukturieren die Analyse des Templates und bilden den Bezugsrahmen für seine spätere Bewertung:

1. Welche fachlich-technischen, qualitativen und betrieblichen Anforderungen werden durch das Technologie-Template adressiert?
2. Wie unterstützen die Architektur und das Profilmodell die Bereitstellung unterschiedlicher Varianten für Web-, Cloud- und Desktop-Projekte?
3. Inwiefern fördern der gemeinsame technische Kern und die Single-Repository-Strategie die Wiederverwendung und Standardisierung zwischen Projekten?
4. Welches Potenzial bietet das Template, wiederkehrende Aufwände bei Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung zu reduzieren?
5. Für welche Projektarten und Kundenanforderungen stellt das Template eine geeignete technische Ausgangsbasis dar?
6. Welche technischen Grenzen, Wartungsaufwände, Risiken und externen Abhängigkeiten sind bei seinem Einsatz zu berücksichtigen?

Die Fragen sind miteinander verknüpft: Aussagen zu Produktivität und Eignung setzen zunächst eine nachvollziehbare Betrachtung der Anforderungen, der Architektur und der vorhandenen Verifikation voraus. Ihre Beantwortung wird daher nicht isoliert, sondern entlang der aufeinander aufbauenden Kapitel der Fallstudie vorgenommen.

1.5 Methodisches Vorgehen

Die Untersuchung ist als technische Fallstudie des implementierten Templates angelegt (Runeson & Höst, 2009). Ausgangspunkt ist eine Bestandsaufnahme des Master Repository. Dabei werden die Repository-Struktur, die enthaltenen Komponenten und die dokumentierten Verantwortungsgrenzen erfasst. Anschließend werden die Architekturentscheidungen und die Beziehungen zwischen Frontend, Backend, Desktop-Integration, gemeinsam genutzten Schnittstellen, Persistenz und Deployment betrachtet. Die Analyse beschreibt den vorhandenen Gegenstand und trennt belegbare Projekteigenschaften von Annahmen oder noch offenen Prüfpunkten (kleiveist, 2026a).

Darauf aufbauend wird das Profilmodell untersucht. Im Mittelpunkt stehen die Abgrenzung der Profile, die Aufteilung zwischen gemeinsamem Kern und profilspezifischen Bestandteilen sowie das Modell optionaler Capabilities. Die Betrachtung des Python-basierten Toolings ergänzt diese strukturelle Analyse um den praktischen Entwicklungsablauf. Hierzu werden insbesondere Projektgenerierung, Systemprüfung, Installation, Entwicklungsbetrieb, Tests, Builds und Release-Vorbereitung als zusammenhängender Workflow betrachtet. Die maßgeblichen Profil- und Werkzeugbeschreibungen werden dabei als projektinterne Primärquellen herangezogen (kleiveist, 2026b, 2026e).

Für die Verifikation werden vorhandene automatisierte Tests, Build-Ergebnisse und dokumentierte Abnahmen den jeweils untersuchten Anforderungen und Architekturentscheidungen zugeordnet. Externe oder produktspezifische Prüfschritte werden davon abgegrenzt. Auf dieser Evidenzbasis erfolgt die Bewertung von Wiederverwendbarkeit, Standardisierung, Produktivitätspotenzial, Stärken und Grenzen. Abschließend werden daraus Kriterien für geeignete Einsatzfelder und für die Übertragung auf zukünftige Kundenprojekte abgeleitet (kleiveist, 2026c).

Abbildung 1 fasst diese aufeinander aufbauenden Untersuchungsschritte zusammen. Die vertikale Darstellung verdeutlicht, dass die Bewertung und der Transfer erst auf Grundlage der zuvor erhobenen und analysierten Projektinformationen erfolgen.

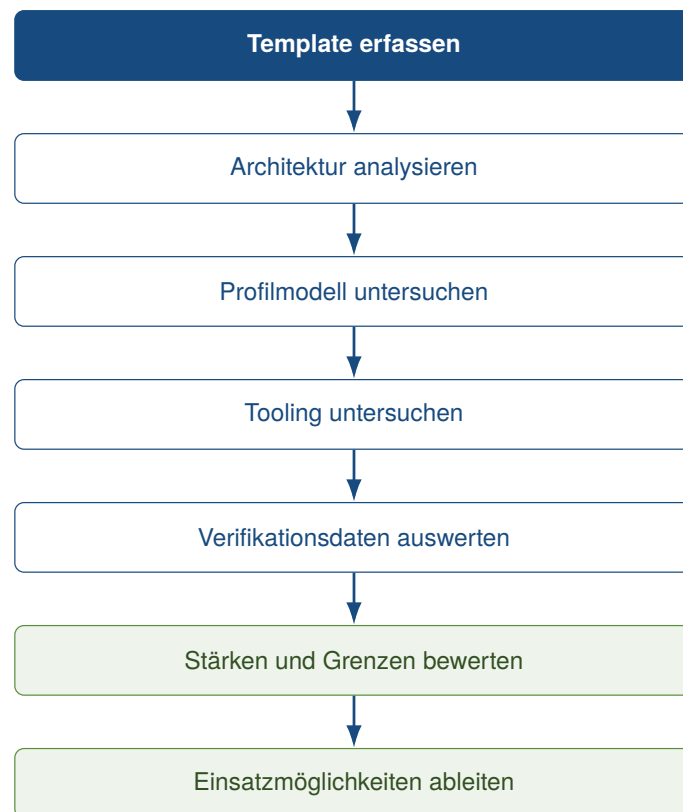


Abbildung 1: Methodische Schritte der Fallstudie

Quelle: Eigene Darstellung.

1.6 Aufbau der Fallstudie

Nach der Einleitung konkretisiert Abschnitt 2 die Anforderungen und die Abgrenzung des Technologie-Templates. Darauf folgt in Abschnitt 3 die Untersuchung seiner Gesamtarchitektur und der beteiligten technischen Komponenten. Abschnitt 4 betrachtet anschließend das Profil- und Feature-Modell einschließlich der optionalen Capabilities und der Single-Repository-Strategie. Das zentrale Projekt-Tooling sowie der Entwicklungs-, Build- und Release-Workflow werden in Abschnitt 5 untersucht.

Abschnitt 6 behandelt die Qualitätssicherung, die technische Verifikation und extern verbleibende Prüfungen. Auf dieser Grundlage bewertet Abschnitt 7 das Template hinsichtlich Produktivitätswirkung, Stärken, Grenzen und Wartungsaufwand. Abschnitt 8 überträgt die gewonnenen Erkenntnisse auf die Auswahl und Generierung zukünftiger Kundenprojekte. Die Arbeit endet mit der Schlussbetrach-

tung in Abschnitt 9, in der die Ergebnisse gebündelt, die Fragestellungen beantwortet und offene Entwicklungsperspektiven eingeordnet werden.

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

Das Template soll eine wiederverwendbare technische Ausgangsbasis für Web-, Cloud- und Desktop-Projekte bilden. Wiederverwendung setzt dabei geplante Gemeinsamkeiten und beherrschte Variabilität voraus (Clements & Northrop, 2001; Krueger, 1992). Projektstruktur, Entwicklungsworkflow, Tooling, Dokumentationsprinzipien, Konfigurationslogik, Tests und Schnittstellen sollen deshalb in einem gemeinsamen Kern zentral gepflegt werden.

Nicht jedes Projekt benötigt sämtliche Komponenten. Profile sollen passende Kombinationen des Kerns auswählen; optionale Capabilities wie PostgreSQL sollen kompatible Backend-Profile unabhängig vom Plattformprofil erweitern (kleiveist, 2026b). Bereits im Zielbild sind Frontend, Backend, Desktop-Schicht, Persistenz, Tooling und Deployment als getrennte Verantwortungsbereiche vorgesehen. Ob die Umsetzung diese Anforderungen erfüllt, wird erst in den nachfolgenden Architektur-, Verifikations- und Bewertungskapiteln geprüft.

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

Einheitlich bedeutet nicht identisch: Die Baseline soll gemeinsame Strukturen und öffentliche Arbeitsabläufe vorgeben, zugleich aber kontrollierte Profilvarianten zulassen. Unabhängige Kopien sind zu vermeiden, weil sie Änderungen des gemeinsamen Fundaments mehrfach erforderlich machen und langfristig auseinanderlaufen können. Tabelle 1 fasst die daraus abgeleiteten, später überprüfbaren Anforderungen zusammen.

Tabelle 1: Zentrale Anforderungen an das Technologie-Template

ID	Anforderung	Operationalisiertes Ziel	spätere Prüfbasis
A-01	Baseline	Grundstruktur muss generierbar sein.	Projektgenerierung
A-02	Struktur	Verantwortlichkeiten liegen an konsistenten Stellen.	Repository-Struktur
A-03	Profile	Nur vorgesehene Komponenten werden aktiviert.	Profilmodell
A-04	Workflows	Prüfen, Initialisieren, Installieren, Starten, Testen und Bauen besitzen gemeinsame Einstiegspunkte.	Tooling
A-05	Wartbarkeit	Gemeinsame Bestandteile werden zentral gepflegt.	Repository-Strategie
A-06	Nachvollziehbarkeit	Änderungen und Konfiguration sind versioniert und dokumentiert.	Versionsstand, Dokumentation
A-07	Erweiterbarkeit	Optionale Fähigkeiten besitzen dokumentierte Abhängigkeiten.	Capability-Modell

Quelle: Eigene Darstellung.

Die Tabelle definiert Soll-Kriterien, keine festgestellten Eigenschaften. Ihre Erfüllung wird anhand der genannten Prüfbasis in den späteren Kapiteln beurteilt.

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

Alle Varianten sollen eine gemeinsame Frontend-Basis, Projektstruktur, Konfigurationslogik sowie ein einheitliches Test- und Dokumentationsmodell nutzen. Browserprojekte benötigen keine native Desktop-Schicht und nur bei Bedarf ein Backend. Cloudfähige Varianten erfordern eine serverseitige API, konfigurierbare Deployment-Grenzen sowie Health- und Readiness-Prüfbarkeit. Desktopvarianten sollen dasselbe Frontend in einer nativen Laufzeit verwenden und Weblogik von plattformspezifischer Integration trennen. Kombinierte Varianten verbinden Desktop-Client, Backend und gegebenenfalls Browserzugriff.

Tabelle 2: Anforderungsmatrix der unterstützten Projektarten

Anforderung	Web	Cloud	Desktop lokal	Desktop + Cloud
Frontend	erforderlich	erforderlich	erforderlich	erforderlich
Backend / API	optional	erforderlich	nicht grundsätzlich	erforderlich
Desktop-Laufzeit	nein	nein	erforderlich	erforderlich
Deployment-Grenze	optional	erforderlich	nein	erforderlich
Persistenz	optional	optional	produktspezifisch	optional

Quelle: Eigene Darstellung.

Die Matrix formuliert Anforderungen, keine Prüfergebnisse. PostgreSQL ist dabei ausschließlich eine optionale Capability für backendfähige Profile und kein eigenes Plattformprofil (kleiveist, 2026b). Öffentliche Client-Konfiguration und serverseitige Werte einschließlich Geheimnissen sind voneinander zu trennen; eine bestimmte Cloud-Plattform wird nicht vorausgesetzt.

2.4 Qualitäts- und Wartbarkeitsanforderungen

Die Qualitätsanforderungen orientieren sich an überprüfbaren Merkmalen des Produktqualitätsmodells nach ISO/IEC 25010 (ISO/IEC, 2023). Komponenten und Profile sollen klare Verantwortlichkeiten besitzen, getrennt testbar und durch dokumentierte Architektur-, Profil- und Workflowentscheidungen wartbar sein. Neue optionale Fähigkeiten sowie Technologieaktualisierungen sollen ohne Duplizierung der gesamten Baseline möglich bleiben; daraus folgt keine Annahme, dass Aktualisierungen automatisch risikofrei sind.

Gleiche Eingaben und Konfigurationen sollen nachvollziehbare Generierungs- und Build-Ergebnisse ermöglichen. Reproduzierbare Builds erhöhen die Prüfbarkeit der Beziehung zwischen Quellstand und Artefakt (Lamb & Zacchioli, 2022). Ferner sollen relevante Zielartefakte kontrolliert erzeugbar, Konfigurationsquellen und Prioritäten verständlich definiert und serverseitige Geheimnisse von Frontend- und Client-Konfiguration getrennt sein. Die tatsächliche Erfüllung wird in Kapitel 6 untersucht.

2.5 Abgrenzung

Das Zielbild ist keine universelle Softwareplattform. Im Mittelpunkt stehen wiederkehrende Web-, Backend-, Desktop- und Full-Stack-Business-Anwendungen. Geschäftsregeln, Domänen- und Kundendatenmodelle sowie produktspezifische Authentifizierungs-, Rollen- und Berechtigungskonzepte gehören nicht zur technischen Baseline. Ebenso wird kein bestimmter Cloud-Provider vorausgesetzt (kleiveist, 2026a).

Stark plattformspezifische native Anwendungen, Spiele und hochspezialisierte Echtzeitsysteme zählen nicht automatisch zum Zielbereich. Dieses Kapitel definiert lediglich Anforderungen und Grenzen; eine Eignungszusage erfolgt erst auf Grundlage der späteren Verifikation und Bewertung.

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

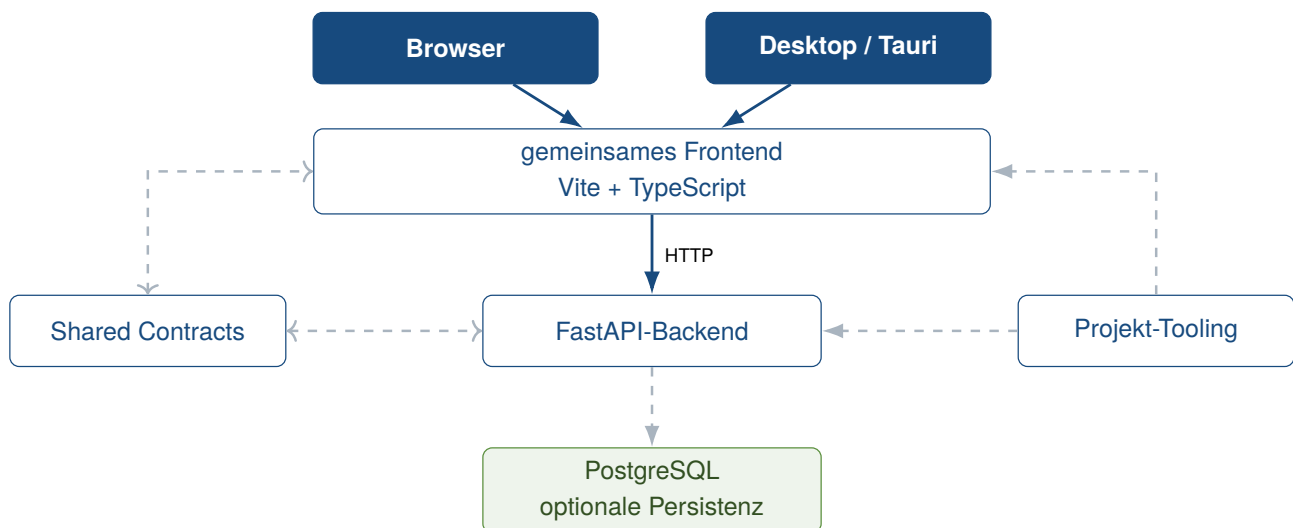


Abbildung 2: Gesamtarchitektur des Technologie-Templates

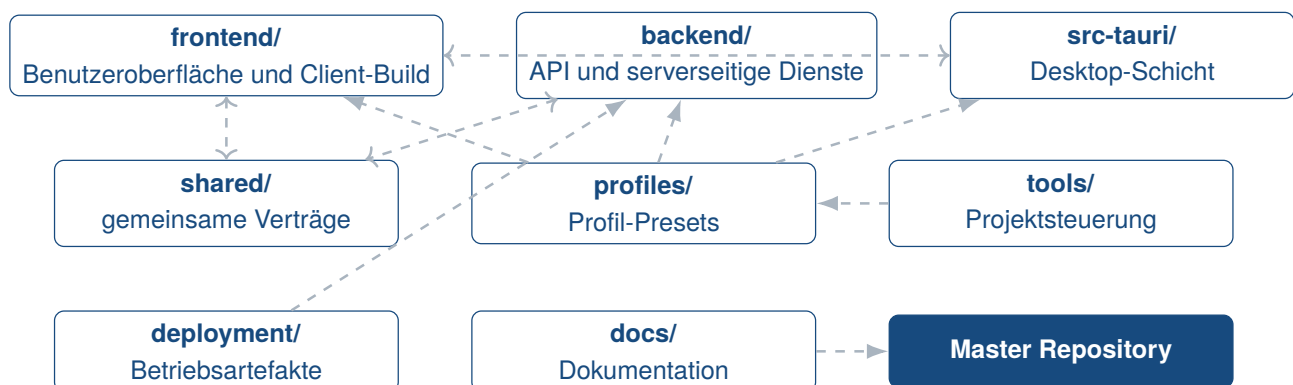
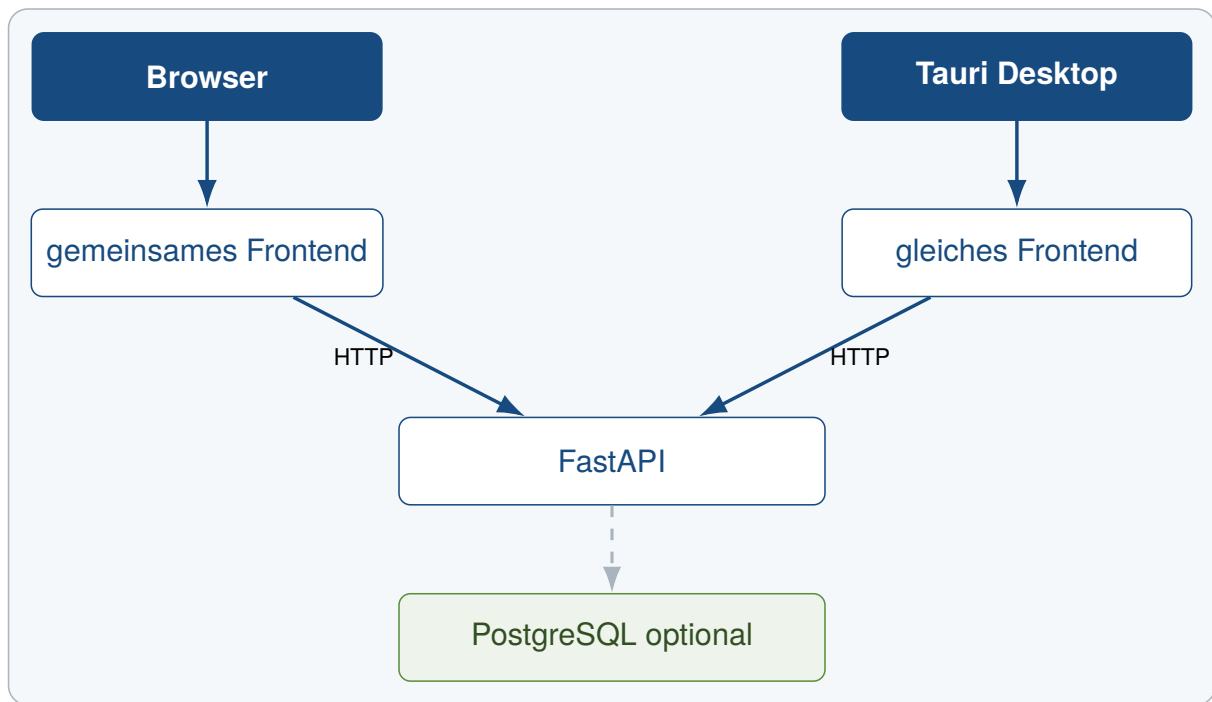


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository



Komponentenbestand abhängig vom gewählten Profil

Abbildung 4: Profilabhängige Laufzeitarchitektur

3.2 Frontend mit Vite und TypeScript

Tabelle 3: Struktur zur Dokumentation des Technologie-Stacks

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
-------------	-------------------	------------------	--------------

3.3 Backend mit FastAPI

3.4 Desktop-Integration mit Tauri und Rust

3.5 Shared Contracts und Schnittstellen

3.6 Konfigurationsmodell

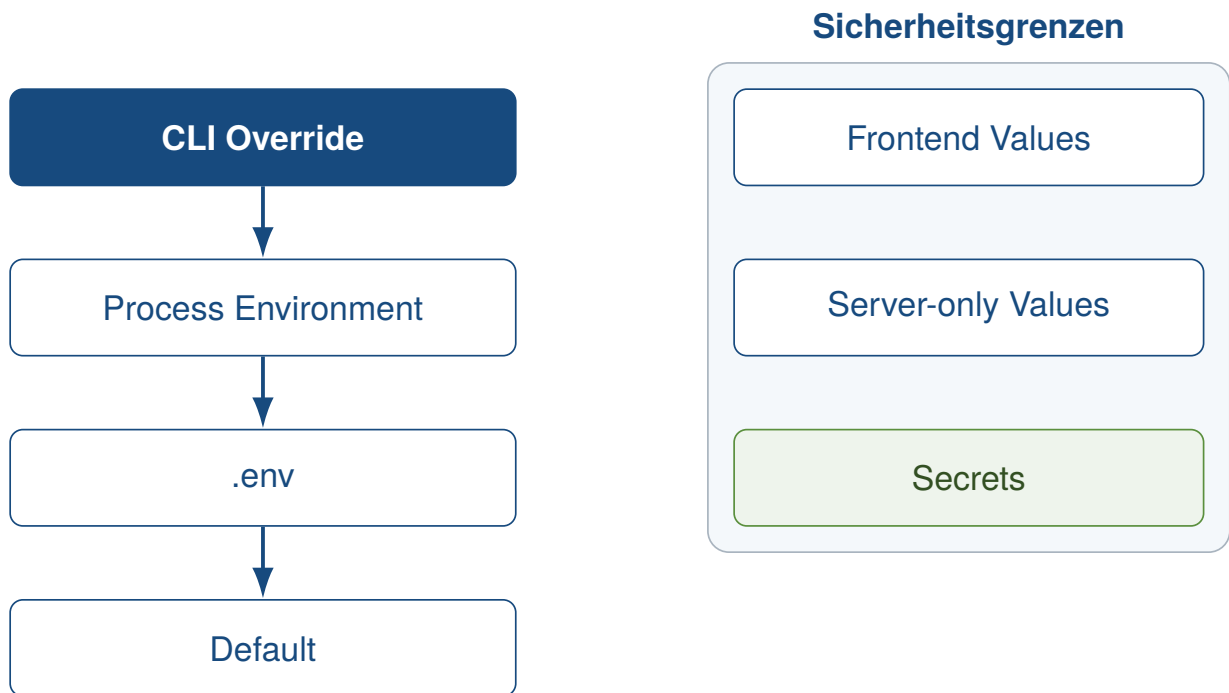


Abbildung 5: Konfigurationspriorität und Sicherheitsgrenzen

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

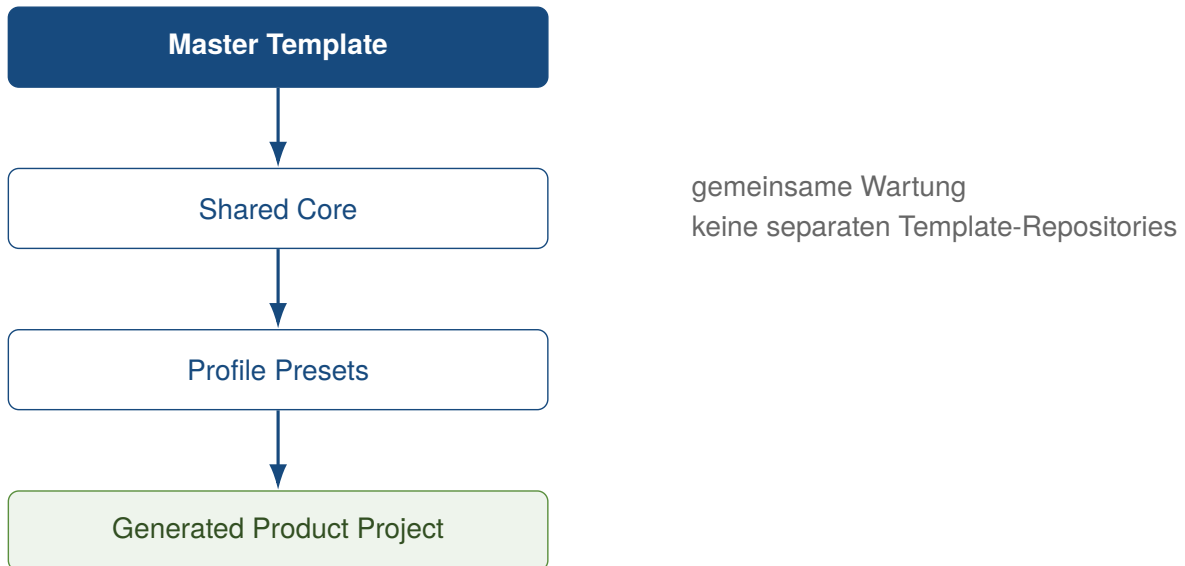


Abbildung 6: Profilmodell auf Basis eines gemeinsamen Master-Templates

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	–
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	–
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 7: Strukturelle Feature-Zuordnung der Projektprofile

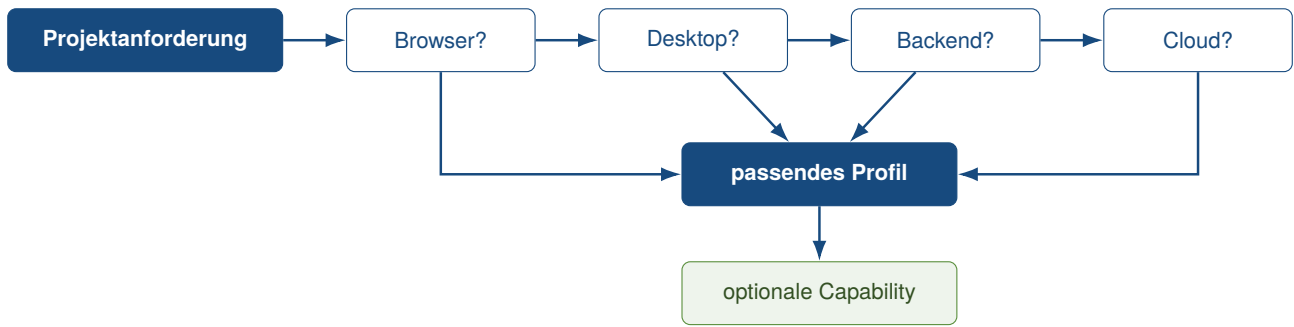


Abbildung 8: Entscheidungsstruktur für die Auswahl eines Projektprofils

Tabelle 4: Struktur zur Dokumentation der Projektprofile

Profil	Plattformbezug	enthaltene Komponenten	optionale Capabilities
--------	----------------	------------------------	------------------------

4.2 Profil Web-only

4.3 Profil Web-cloud

4.4 Profil Desktop-local

4.5 Profil Desktop-cloud

4.6 Profil Full-platform

4.7 Optionale Capabilities

4.8 Single-Repository-Strategie

5 Tooling und Entwicklungsworkflow

5.1 Rolle von control.py

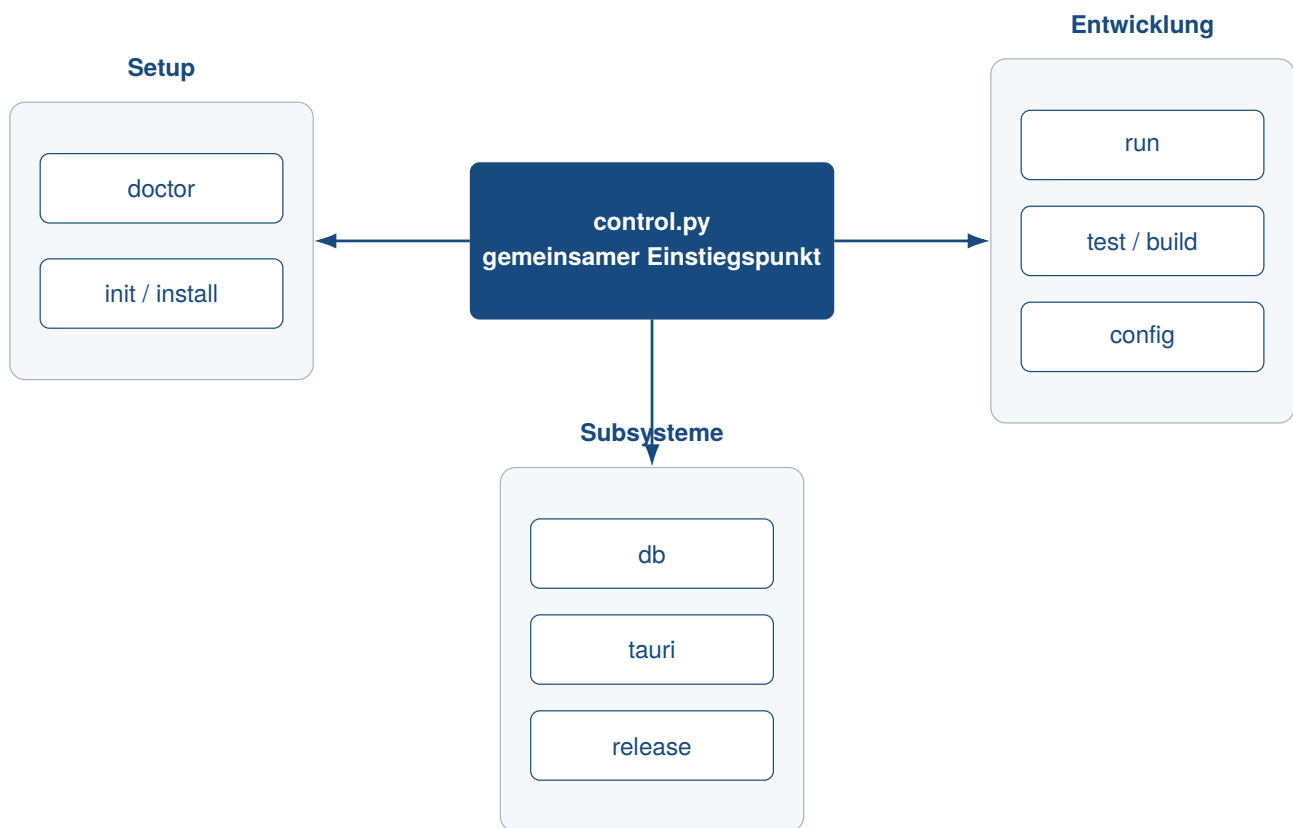


Abbildung 9: Befehlsgruppen des zentralen Projekt-Toolings

5.2 Projektinitialisierung und Generierung

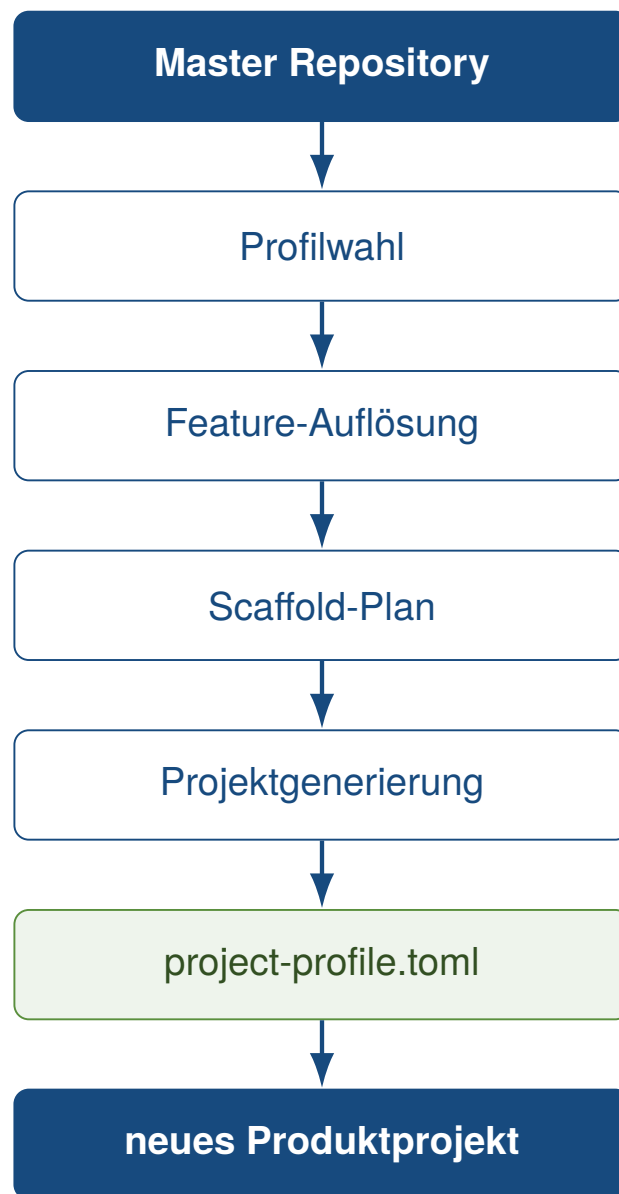


Abbildung 10: Workflow der profilgesteuerten Projektgenerierung

5.3 Systemprüfung und Installation

5.4 Entwicklungsbetrieb



Abbildung 11: Grundstruktur des Entwicklungsworkflows

5.5 Tests und Builds

5.6 Release- und Packaging-Workflow

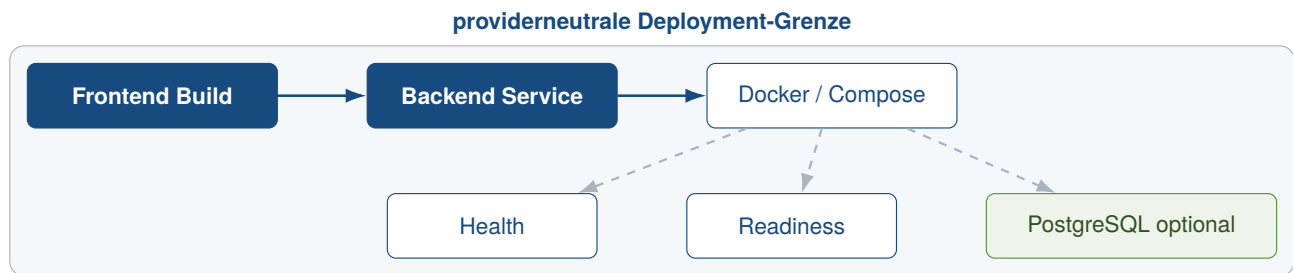


Abbildung 12: Providerneutrale Deployment-Architektur

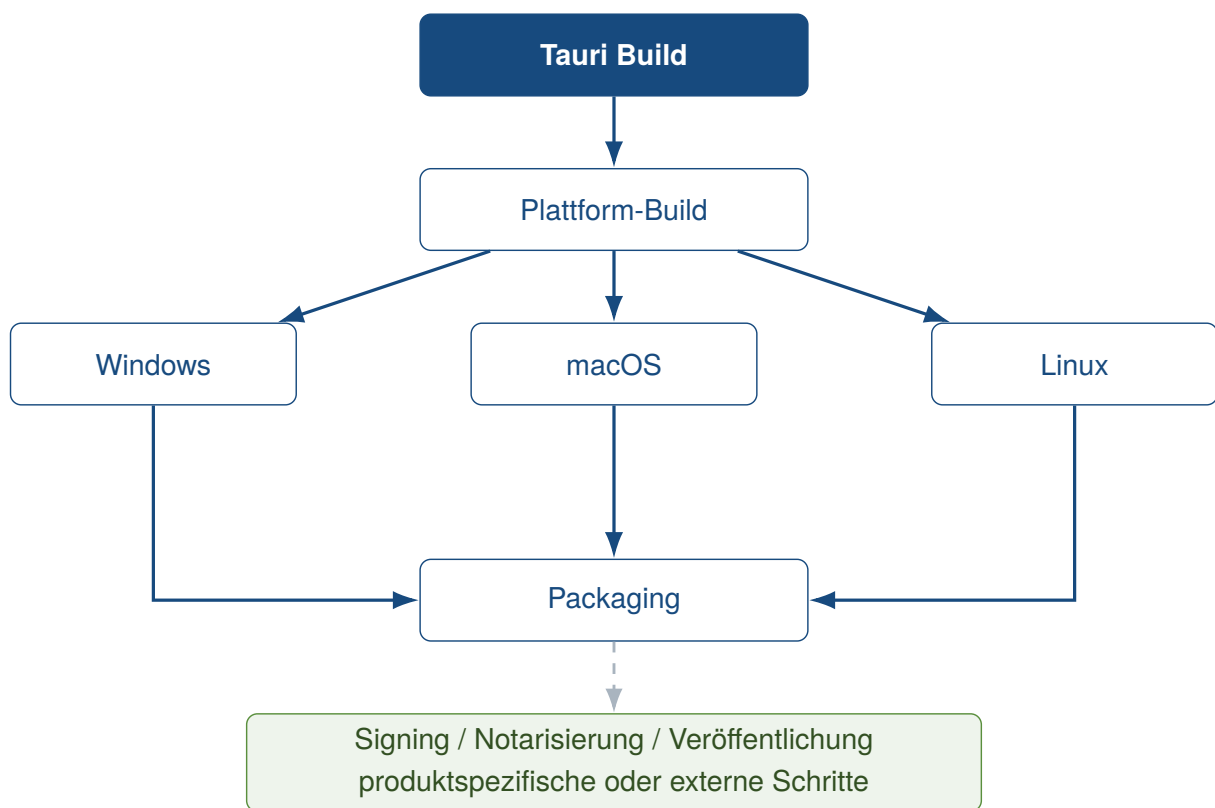


Abbildung 13: Plattformübergreifendes Modell für Desktop-Releases

5.7 Entwickler-Onboarding

6 Qualitätssicherung und Verifikation

6.1 Teststrategie



Abbildung 14: Nachweiskette der technischen Verifikation

6.2 Continuous Integration

6.3 Abnahme und technische Verifikation

Profil	Generate	Install	Doctor	Tests	Web Build	Desktop / Tauri	Database	Container
web-only	offen	offen	offen	offen	offen	offen	offen	offen
web-cloud	offen	offen	offen	offen	offen	offen	offen	offen
desktop-local	offen	offen	offen	offen	offen	offen	offen	offen
desktop-cloud	offen	offen	offen	offen	offen	offen	offen	offen
full-platform	offen	offen	offen	offen	offen	offen	offen	offen

Abbildung 15: Struktur der profilbezogenen Verifikationsmatrix

Tabelle 5: Struktur zur Dokumentation der technischen Verifikation

Profil	Prüfbereich	Kriterium	Evidenz	Ergebnis
--------	-------------	-----------	---------	----------

6.4 Reproduzierbarkeit

6.5 Security und Konfigurationsgrenzen

6.6 Extern verbleibende Prüfungen

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

7.2 Stärken

Tabelle 6: Struktur zur Gegenüberstellung von Stärken und Grenzen

Aspekt	Stärke / Evidenz	Grenze / Evidenz	Auswirkung
--------	------------------	------------------	------------

7.3 Schwächen und Grenzen

7.4 Wartungsaufwand

7.5 Vergleich mit alternativen Template-Strategien

7.6 Gesamtbewertung

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Projekttyp	Anforderungsfit	Anpassungsbedarf	Evidenz
Web-Anwendungen	offen	offen	offen
Cloud-Anwendungen	offen	offen	offen
Desktop-Anwendungen	offen	offen	offen
SaaS + Desktop	offen	offen	offen
lokale Business-Tools	offen	offen	offen
stark native Anwendungen	offen	offen	offen
Echtzeitsysteme	offen	offen	offen
Spiele	offen	offen	offen

Abbildung 16: Offene Bewertungsstruktur für die Projekteignung

Tabelle 7: Struktur zur Bewertung der Projekteignung

Projekttyp	Anforderungsfit	Anpassungsbedarf	Grenzen	Evidenz
------------	-----------------	------------------	---------	---------

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

8.3 Analyse von Kundenanforderungen

8.4 Auswahl des Projektprofils

8.5 Generierung eines Kundenprojekts



Abbildung 17: Prozess vom Kundenbedarf bis zur Auslieferung

8.6 Überführung in die Produktentwicklung

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

9.2 Beantwortung der Fragestellungen

9.3 Kritische Würdigung

9.4 Ausblick

9.5 Fazit

Literaturverzeichnis

- Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Verfügbar 8. August 2026 unter <https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/>
- ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard Nr. ISO/IEC 25010:2023). International Organization for Standardization. Verfügbar 8. August 2026 unter <https://www.iso.org/standard/78176.html>
- kleiveist. (2026a, 6. August). *Framework architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026b, 6. August). *Project profiles* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md>
- kleiveist. (2026c, 7. August). *Template final acceptance* [Technisches Abnahmeprotokoll im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026d, 7. August). *Template-projekte: Full-stack project template* [GitHub-Repository, geprüfter Commit a4487ac]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026e, 6. August). *Tooling guide* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md>
- Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, 24(2), 131–183. <https://doi.org/10.1145/130844.130856>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073045>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>